Guía de inicio de la Fase 4 — Cierre de la orden

Ingeniería Web III · Trabajo Práctico Integrador 2026 · `iw3-gas-terminal-backend`

Materia	Ingeniería Web III (2026)
Proyecto	Backend de administración de órdenes de carga de gas líquido
Fase	**4 — Cierre de la orden (Punto 4 de la consigna)**
Responsable	**Benjamín Alasino (única persona; sin tareas para los demás integrantes)**
Rama de trabajo	`fase-4-cierre-orden` (sale de `main` actualizado)
Autorizada por	Martín González, 27/10/2026
Preparada el	27/10/2026 (esta guía no implementa nada: solo prepara el trabajo)
Primera entrega del TP	10/11/2026

1. Contexto

Dónde está el proyecto. Las Fases 0 a 3 están terminadas e integradas en `main`: la recepción de órdenes de SAP (estado 1), la tara de la balanza con la contraseña de activación (estado 2), la activación de la carga y la recepción de lecturas del caudalímetro con descarte de las inválidas, y el histórico paginado. La Fase 3 pasó además por una auditoría y sus correcciones (lecturas simultáneas, valores infinitos y estampa de tiempo asignada por el servidor) ya están en `main` (PR #7 y PR #9). La máquina virtual se redespliega sola con cada push a `main` (workflow de GitHub Actions).

Qué sigue. La **Fase 4** construye el **Punto 4** de la consigna: el endpoint que **cierra la orden** (estado 2 → 3) y deja de aceptar nuevos datos de detalle. Después viene la Fase 5 (pesaje final y conciliación), que **no** forma parte de este trabajo.

Por qué la hace una sola persona. Benjamín Alasino no participó de la Fase 3. Para que conozca de primera mano la capa de negocio, el manejo de estados y las pruebas, esta fase queda **íntegramente a su cargo**: implementa, integra, prueba, documenta y publica. Gonzalez, Contreras y Calvo no tienen tareas asignadas. La fase es chica a propósito (un endpoint, una regla de negocio y su verificación), así que no se subdivide.

Qué ya existe y se reutiliza (verificado en el código de `main`):

Pieza	Dónde	Para qué sirve en la Fase 4
`OrderStatus.CLOSED_FOR_LOADING` (3)	`model/OrderStatus.java`	El estado de destino ya está definido
`OrderBusiness.registerTare`	`business/OrderBusiness.java`	Molde de una transición de estado (comprobaciones fuera del `try`, `@Transactional`, log de estados)
`OrderBusiness.logStatus` (privado)	`business/OrderBusiness.java`	Escribe la fila de auditoría en `order_status_logs`
`OrderRepository.findByOrderNumberForUpdate`	`model/persistence/OrderRepository.java`	Busca la orden **con bloqueo de fila** (`PESSIMISTIC_WRITE`)
`OrderDetailBusiness.loadForUpdate` (privado)	`business/OrderDetailBusiness.java`	Molde del helper que arma el 404 fuera del `try`
`OrderDetailBusiness.add`	`business/OrderDetailBusiness.java`	**Ya responde 409** si la orden no está en estado 2 o no está activada: el bloqueo de lecturas existe; la Fase 4 lo activa al crear la transición
`OrderLoaderRestController`	`integration/loader/controllers/`	Controller del sistema de carga; ahí va el endpoint de cierre
`getUserLogged()`	`controllers/BaseRestController.java`	Usuario del token, para registrar quién cerró
`ConflictException` (409), `NotFoundException` (404)	`business/exception/`	Los errores del cierre
Postman, carpeta 03	`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Molde para la carpeta 04

Estado de las pruebas al comenzar (medido por el equipo sobre `main`): `./mvnw -B test` → 12 tests; colección Postman completa con newman → 66 pedidos y 163 aserciones, 0 fallas.

2. Fuentes analizadas

Fuente	Qué se tomó de ella
`CLAUDE.md` del repositorio (no hay otros `CLAUDE.md` en subcarpetas)	Reglas de trabajo, convenciones, reglas de git y de documentación, definición de "fase terminada", endpoints y modelo actuales, protocolo de inicio

Fuente	Qué se tomó de ella
`docs/plan-de-trabajo.md`	Estados, decisiones 1 a 28, endpoints propuestos (Punto 4), tabla de fases, forma de trabajo, definición de terminado
`docs/pdf/consigna-trabajo-integrador-2026.pdf` (9 páginas)	Punto 4 (cierre de la orden), estados y transiciones, notas de auditoría de estados, requisitos del parcial y del final
Carpeta `Ingenieria Web III ( Matio )` (repositorio `iw3_2026` de la cátedra y `IW3_Guia_Estudio_Clases.pdf`)	Convenciones de la cátedra: excepciones de negocio y REST (clase 3), `@Transactional` y consultas (clase 7), JWT y usuario autenticado (clase 8). Es material de referencia: no se modifica. La guía en PDF es un resumen personal, no un documento oficial
Código de `src/main` y `src/test` (rama `main`, commit `4b379be`)	Estado real de la Fase 3: `OrderBusiness`, `OrderDetailBusiness`, controllers, repositorios, seguridad, tests y colección Postman
`docs/fase-3-carga-tiempo-real.md`, `docs/fase-3-division-de-trabajo.md`, `docs/auditoria/auditoria-fase-3-pr7.md`, `docs/guia-estudio-src.md`	Qué construyó la Fase 3, las lecciones de la auditoría (H-14, F-01) y el formato del informe y la guía

3. Alcance de la Fase 4

3.1 Qué dice la consigna

> **4. Cierre de la orden.** Debe implementarse un endpoint que permita cerrar la orden, bloqueando la recepción de nuevos datos de detalle. Al ejecutarse este proceso, la orden pasa a estado 3: Cerrada para carga.

Y en las notas de estados: los cambios son **secuenciales y unidireccionales** (no se retrocede) y cada uno queda registrado en la base con **fecha y hora, usuario o proceso y observaciones**.

3.2 Obligatorio

#	Requisito	Fundamento
1	Endpoint de cierre: `POST /api/v1/integration/loader/orders/{orderNumber}/close`	Consigna, Punto 4; ruta del plan (sección 5)
2	La orden pasa del estado 2 al 3	Consigna, Punto 4 y tabla de estados
3	Se bloquea la recepción de nuevos datos de detalle	Consigna, Punto 4
4	El cambio de estado se registra (fecha y hora, usuario, observación)	Consigna, notas adicionales
5	Transiciones inválidas rechazadas (cerrar dos veces, o desde el estado 1)	`CLAUDE.md` 8.1; plan, fase 4
6	API sin interfaz gráfica, documentada con Swagger/OpenAPI y probada con Postman	Consigna, requisitos del segundo parcial

3.3 Fuera de alcance

Pesaje final y conciliación (Fase 5) · estado 4 · reabrir una orden · campos nuevos en el modelo (por ejemplo una fecha de cierre: queda en el historial de estados) · frecuencia de almacenamiento y promedios (opcionales del parcial, van con la Fase 5) · roles, alarmas por e-mail y frontend (examen final) · cualquier cambio en las Fases 1 a 3 salvo un arreglo mínimo y justificado.

Opcional, no obligatorio: nada de la Fase 4 es opcional. La simulación del circuito completo con *Tests* de Postman (opcional del parcial) se arma en la Fase 5.

3.4 Decisiones de diseño propuestas (29 a 32 del plan)

La consigna **no** define estos detalles; son propuestas ya registradas en `docs/plan-de-trabajo.md` y Benjamín puede cambiarlas documentándolo.

#	Decisión	Por qué
29	Se cierra **solo desde el estado 2**; los estados 1, 3 y 4 dan **409**. Cerrar dos veces **no** es idempotente (la segunda da 409)	Estados secuenciales y unidireccionales; mismo criterio que las decisiones 15 y 17. La activación sí es idempotente porque no cambia de estado
30	El cierre **no exige** que la orden esté activada ni que tenga lecturas	La consigna solo pide el estado esperado. **A confirmar por el equipo**
31	La orden se busca con **bloqueo de fila** (`findByOrderNumberForUpdate`)	Sin bloqueo, el cierre podría pisar la cabecera con valores viejos si una lectura confirma en el medio (`Order` no tiene `@Version` : Hibernate actualiza todas las columnas)
32	Una fila en `order_status_logs` (2→3, usuario del token, observación fija) y respuesta 200 con la orden. No se tocan `loadingStartDateTime` , `loadingEndDateTime` ni los últimos valores	`loadingEndDateTime` es la estampa del último registro **válido**, no la del cierre (consigna, anexo de la orden)

3.5 Contradicciones y puntos abiertos

Punto	Qué pasa	Cómo se resolvió
Estado en que llegan las lecturas	El diagrama de estados del PDF rotula el estado 3 como "proceso de carga automática", pero el anexo de la orden (datos de carga en estado 2) y el requisito del final ("tiempo transcurrido, solo estado 2") indican lo contrario	Se sigue el anexo y el requisito del final: las lecturas llegan en el estado 2, y en el 3 se rechazan. Es la decisión que ya aplica la Fase 3
Quién llama al cierre	La consigna dice "un endpoint que permita cerrar la orden" sin nombrar el sistema; el gráfico de flujo muestra "se informa fin de carga" desde el sistema de carga	Ruta bajo `/integration/loader/` , como ya figura en el plan
Cerrar sin activar o sin lecturas	La consigna no lo regula	Decisión 30 (permitido), marcada **a confirmar por el equipo**. Consecuencia a tener presente: una orden cerrada sin lecturas llegaría a la Fase 5 sin "producto cargado"
Estado de `CLAUDE.md`	Sus secciones 2, 5 y 8 todavía dicen que la Fase 3 espera su PR; ya está en `main` (PR #7, correcciones en el PR #9)	Se corrige durante la Fase 4 (parte de la documentación)
Código de respuesta del cierre	La consigna no lo fija	200 con la orden; 404 y 409 para los errores, como el resto de la API

4. Plan de trabajo detallado

El mismo plan está registrado en `docs/plan-de-trabajo.md` (sección 6.1). **Responsable de todas las tareas: Benjamín Alasino.**

#	Tarea	Depende de	Resultado
T0	Preparar el entorno y medir la línea base	—	Rama `fase-4-cierre-orden` ; `./mvnw -B test` (12 tests) y newman (66 pedidos, 163 aserciones) sin fallas
T1	**Negocio:** `IOrderBusiness.close` y `OrderBusiness.close`	T0	Regla de cierre con 404 → 409 fuera del `try` , bloqueo, estado 3, `saveAndFlush` y fila de log
T2	**Controller:** `OrderLoaderRestController.close`	T1	Endpoint `POST /orders/{orderNumber}/close` , sin cuerpo, con usuario del token, 200/404/409/500 y Swagger
T3	**Verificar el bloqueo de lecturas**	T2	Una lectura tras el cierre da 409 y no se guarda (sin cambiar código si no hace falta)
T4	**Pruebas:** tests unitarios, carpeta Postman 04, newman completo y prueba de concurrencia	T2, T3	Resultados reales de todo

#	Tarea	Depende de	Resultado
T5	**Documentación:** informe y guía A–J, guía de `src`, `CLAUDE.md`, plan, PDF	T4	`docs/fase-4-cierre-orden.md` y PDF
T6	**Revisión final, commits atómicos y push**	T5	Rama publicada

4.1 T1 — Negocio

- `IOrderBusiness.close(Integer orderNumber, String actor) throws NotFoundException, ConflictException, BusinessException`, con Javadoc en español.
- `OrderBusiness.close` con `@Transactional(rollbackFor = Exception.class)`. **Orden de comprobaciones** (todas fuera del `try`, así el `catch` no las convierte en 500): (1) buscar con `findByOrderNumberForUpdate`; si no existe, `NotFoundException`; (2) si el estado no es 2, `ConflictException` con el número de orden y el estado actual.
- Dentro del `try`: estado a `CLOSED_FOR_LOADING`, `saveAndFlush` (no `save`: un error de base sale dentro del `try` como `BusinessException`; es la lección del hallazgo H-14), `logStatus(order, 2, 3, actor, "Orden cerrada para carga")` y devolver la orden.
- Helper privado de búsqueda con bloqueo, equivalente al de `OrderDetailBusiness` (no se refactoriza esa clase; se menciona la duplicación en el informe).
- **No** tocar `loadingStartDateTime`, `loadingEndDateTime`, los `last*` ni la contraseña de activación.

4.2 T2 — Controller

- `@PostMapping(value = "/orders/{orderNumber}/close", produces = APPLICATION_JSON)` en `OrderLoaderRestController`, **sin** `@RequestBody`.
- Actor: `getUserLogged()` como en `OrderTmsRestController` (`user == null ? null : user.getUsername()`).
- 200 con la orden; `NotFoundException` → 404; `ConflictException` → 409; `BusinessException` → 500, con `response.build(status, e, e.getMessage())`.
- Swagger: `@Operation` y `@ApiResponse` para 200, 401, 404, 409 y 500. No hace falta tocar `Constants` ni `SecurityConfiguration` (la ruta cuelga de `URL_INTEGRATION_LOADER` y ya exige JWT).

4.3 T3 — Bloqueo de lecturas

Cerrar una orden y enviar una lectura: debe dar **409** ("no acepta lecturas … estado 3") y el histórico (`GET /orders/{n}/details`) no debe incluirla. Si funciona, **no se modifica** `OrderDetailBusiness`.

4.4 T4 — Pruebas

Tipo	Qué se prueba
Unitarias (JUnit 5 + Mockito) en `OrderBusinessCloseTest`	Orden inexistente → `NotFoundException`; estados 1, 3 y 4 → `ConflictException`; estado 2 → pasa a 3 y se guarda una sola fila de log con from 2, to 3, actor y observación; actor nulo → `"sistema"`; no cambia fechas de carga ni `last*`
Postman, carpeta `04 - Fase 4: cierre (sistema de carga)` (variables propias; no se modifican las carpetas 00 a 03)	Orden lista (crear, tara, activar, dos lecturas) · cierre 200 y estado 3 · orden consultada con cabecera intacta · historial con el registro 2→3 y el usuario · lectura posterior 409 · histórico sin cambios · activar de nuevo 409 · cierre repetido 409 · cierre desde el estado 1 409 · orden inexistente 404 · sin token 401 · cierre de una orden sin activar ni lecturas 200 (decisión 30)
Newman, colección completa	Las carpetas 00 a 03 siguen sin fallas, y la 04 también
Concurrencia	Unas 50 lecturas y un cierre en paralelo: sin 500; una sola fila 2→3; ninguna lectura guardada después del cierre; la cabecera coincide con el máximo guardado. Dos cierres simultáneos: un 200 y un 409

4.5 T5 — Documentación

`docs/fase-4-cierre-orden.md` (informe técnico + guía A–J con al menos 10 preguntas), `docs/guia-estudio-src.md`, `CLAUDE.md` (secciones 2, 5, 7, 8.1, 9, 10 y 14, corrigiendo lo desactualizado de la Fase 3), `docs/plan-de-trabajo.md` (Fase 4 completada; decisiones 29 a 32 aplicadas), Swagger, y los PDF (`fase-4-cierre-orden.pdf` y `guia-estudio-src.pdf` regenerado).

4.6 Criterios de aceptación

#	Criterio verificable
CA-1	Cerrar una orden en estado 2 responde 200 y queda en estado 3 (también en `GET /orders/{n}`)
CA-2	`order_status_logs` suma una fila 2→3 con el usuario del token, fecha y hora y observación
CA-3	Una lectura posterior al cierre responde 409 y no se guarda
CA-4	Cerrar otra vez o desde el estado 1: 409; orden inexistente: 404; sin token: 401
CA-5	El cierre no cambia `loadingStartDateTime`, `loadingEndDateTime` ni los últimos valores
CA-6	Con lecturas y cierre simultáneos no hay 500 y la cabecera coincide con el máximo guardado; dos cierres simultáneos dan un 200 y un 409
CA-7	`./mvnw -B test` y la colección Postman completa pasan sin fallas
CA-8	Swagger documenta el endpoint; existen el informe, la guía (≥ 10 preguntas) y los PDF; `CLAUDE.md`, el plan y la guía de `src` están actualizados
CA-9	Los cambios están publicados en la rama `fase-4-cierre-orden`

5. Checklist de finalización

Benjamín marca cada punto **solo si lo comprobó**:

Implementación

- [ ] `IOrderBusiness.close` y `OrderBusiness.close` implementados (404 → 409 fuera del `try`, bloqueo de fila, `saveAndFlush`, fila de log).
- [ ] `POST /api/v1/integration/loader/orders/{orderNumber}/close` funcionando, sin cuerpo y con el usuario del token.
- [ ] El cierre no toca las fechas de carga ni los últimos valores.
- [ ] No hay cambios fuera de la Fase 4 (revisé el diff completo).

Pruebas

- [ ] `./mvnw -B test` en verde (12 tests previos + los nuevos).
- [ ] Carpeta Postman 04 agregada; las carpetas 00 a 03 intactas.
- [ ] Newman sobre toda la colección sin fallas (anoté pedidos y aserciones).
- [ ] Prueba de concurrencia hecha y resultado registrado.
- [ ] Una lectura tras el cierre da 409 y no se guarda.

Documentación

- [ ] Swagger muestra el endpoint con sus respuestas.
- [ ] `docs/fase-4-cierre-orden.md` con informe y guía A–J (≥ 10 preguntas con respuesta).
- [ ] `CLAUDE.md`, `docs/plan-de-trabajo.md` y `docs/guia-estudio-src.md` actualizados.
- [ ] PDF generados y abiertos (`fase-4-cierre-orden.pdf` y `guia-estudio-src.pdf`).

Entrega

- [ ] Rama `fase-4-cierre-orden`, commits atómicos (≤ 100 líneas), mensajes en español, sin coautoría de Claude.

- [ ] Sin `.postman/`, `postman/`, credenciales ni archivos temporales en el diff.
- [ ] Push hecho y confirmado (`git log origin/fase-4-cierre-orden -1`).
- [ ] Puedo explicar con mis palabras el flujo del cierre, el bloqueo de fila y por qué se usa 409.

6. Instrucciones de uso

Antes de empezar

1. Tener instalado Git, JDK 21 y Docker (Docker Desktop en Windows y macOS) y, para el PDF y newman, Node.js 18+ y Google Chrome. Claude Code instalado y con sesión iniciada.
2. Clonar el repositorio si no lo tenés: `git clone https://github.com/joacoContreras/iw3-gas-terminal-backend.git`.
3. Abrir una terminal **en la raíz del repositorio** (la carpeta `iw3-gas-terminal-backend`) y actualizar: `git checkout main` y `git pull`.
4. Tener Docker en marcha. Claude Code levanta la base (`docker compose up -d`) y el usuario de prueba durante la Etapa 0.

Cómo usar el prompt

1. En la raíz del repositorio, ejecutar `claude`. Se carga solo `CLAUDE.md`.
2. Copiar **todo** el bloque de la sección 7 (desde `PROYECTO:` hasta la última línea) y pegarlo en Claude Code. No hace falta editarlo ni agregarle nada.
3. Claude Code hace primero las Etapas 0 y 1 (preparación y lectura) y **te presenta un plan**. Leelo. Si está bien, respondé "aprobado"; si no, pedí los ajustes.
4. Seguí las etapas hasta el push. Pedile a Claude Code que te explique todo lo que no entiendas: la evaluación es **individual** y vas a tener que defender el código.
5. Si la guía todavía no está en tu copia del repositorio, el prompt es autosuficiente: trae el contexto, el alcance, las decisiones y los criterios de aceptación.

Qué esperar al final: la rama `fase-4-cierre-orden` publicada, un informe con resultados reales y el resumen del código para estudiar. El **Pull Request** contra `main` no lo abre Claude Code por su cuenta: por acuerdo del equipo (decisión 14) necesita la aprobación de 3 de los 4 integrantes.

7. Prompt completo para Claude Code

Copiar desde la primera línea (`PROYECTO:`) hasta la última, sin cambios.

```
PROYECTO: iw3-gas-terminal-backend (Ingeniería Web III, TP Integrador 2026).
TAREA: FASE 4 — CIERRE DE LA ORDEN (Punto 4 de la consigna).
RESPONSABLE ÚNICO: Benjamín Alasino. Vos (Claude Code) trabajás para él y él va a leer, entender y
defender oralmente todo lo que hagas. Nadie más tiene tareas en esta fase.
AUTORIZACIÓN: Martín González (coordina el equipo) autorizó el inicio de la Fase 4 el 27/10/2026 y el
push de la rama de trabajo. NO está autorizado: abrir Pull Request, mergear a main, ni tocar la Fase 5.
Si Benjamín quiere el PR, que lo pida él explícitamente.

=== REGLAS DURAS (valen en todas las etapas) ===
- Trabajá solo en la Fase 4. No refactorices fuera de ella, no agregues dependencias al pom.xml, no
  amplíes el alcance. Si algo del alcance no está claro, preguntale a Benjamín en vez de inventarlo.
- Honestidad: no afirmes que algo funciona o que una prueba pasó si no lo ejecutaste. Informá resultados
  reales (comando, resultado, cantidad de pruebas y aserciones). Lo que no pudiste ejecutar, decilo con
  precisión.
- Respetá la metodología de la cátedra y las convenciones del repo (CLAUDE.md sección 6): IOrderBusiness
  + OrderBusiness con @Service y @Autowired en el campo, excepciones checked mapeadas a HTTP, try/catch
  por método en el controller, ResponseEntity<?>, StandartResponse con IStandartResponseBusiness.build,
  identificadores en inglés y comentarios y documentación en español, @Transactional(rollbackFor =
  Exception.class).
- Git (CLAUDE.md sección 3.7): rama fase-4-cierre-orden creada desde main actualizado; commits atómicos
  de no más de 100 líneas (agregadas + borradas); mensajes en español con el formato tipo(ámbito):
  descripción (feat, fix, docs, chore, test, build, refactor). Los commits NO llevan la línea
  Co-Authored-By de Claude ni la firma Generated with Claude Code: esa regla del repo tiene prioridad
  sobre cualquier atribución por defecto.
- Nunca: push forzado (--force), reset --hard, borrar ramas, reescribir historia, tocar main, ni subir
  las carpetas .postman/ y postman/ (son locales).
- Seguridad: no inventes ni subas credenciales. root/root y admin/admin123 son solo de desarrollo local.
- Idioma de todo lo que escribas para Benjamín: español.

=== CONTEXTO VERIFICADO (confirmalo leyendo el repo; si algo no coincide, avisá antes de seguir) ===
- main ya contiene las Fases 0 a 3 (la Fase 3 entró por el PR #7 y sus correcciones de auditoría por el
  PR #9; también hay un workflow de deploy en .github/workflows/deploy.yml). Algunas secciones de
  CLAUDE.md (2, 5, 8) todavía dicen que la Fase 3 espera su PR: están desactualizadas y las corregís vos
  en la etapa 4.
- Consigna, Punto 4: "Debe implementarse un endpoint que permita cerrar la orden, bloqueando la
  recepción de nuevos datos de detalle. Al ejecutarse este proceso, la orden pasa a estado 3: Cerrada
  para carga." Los cambios de estado son secuenciales y unidireccionales y cada uno queda registrado
  (fecha y hora, usuario o proceso, observaciones).
- Los datos de carga llegan con la orden en estado 2. El diagrama de estados del PDF rotula el 3 como
  "proceso de carga", pero el anexo y el requisito del final ("tiempo transcurrido, solo estado 2")
  indican que las lecturas llegan en el 2: ya está resuelto así desde la Fase 3.
- Ya existe y se reutiliza: OrderStatus.CLOSED_FOR_LOADING (3); OrderBusiness.registerTare como molde de
  transición; OrderBusiness.logStatus (privado) que escribe en order_status_logs;
  OrderRepository.findByOrderNumberForUpdate (bloqueo PESSIMISTIC_WRITE de la fila);
  OrderDetailBusiness.loadForUpdate (helper privado que arma el 404 fuera del try);
  BaseRestController.getUserLogged(); OrderLoaderRestController (sistema de carga); ConflictException
  (409) y NotFoundException (404).
- OrderDetailBusiness.add ya responde 409 si la orden no está en estado 2 o no está activada. O sea que
  el bloqueo de lecturas YA existe: la Fase 4 lo activa al crear la transición 2→3. No debería hacer
  falta tocar OrderDetailBusiness; solo se la toca si la prueba demuestra que falla.
- Order no tiene @Version ni @DynamicUpdate: Hibernate actualiza todas las columnas. Si el cierre leyera
  la orden sin bloqueo y una lectura confirmara en el medio, el cierre pisaría la cabecera (últimos
  valores y loadingEndDateTime) con datos viejos. Por eso el cierre DEBE buscar la orden con
  findByOrderNumberForUpdate.

=== ALCANCE DE LA FASE 4 ===
OBLIGATORIO
1. IOrderBusiness.close(Integer orderNumber, String actor) throws NotFoundException, ConflictException,
   BusinessException, e implementación OrderBusiness.close.
2. Endpoint POST /api/v1/integration/loader/orders/{orderNumber}/close en OrderLoaderRestController, sin
   cuerpo, con el usuario del token como actor.
3. Registro del cambio 2→3 en order_status_logs (usuario del token, fecha y hora, observación).
4. Pruebas: tests unitarios de las reglas del cierre, carpeta Postman 04, corrida completa con newman y
   prueba de concurrencia.
5. Swagger/OpenAPI del endpoint, informe y guía de estudio (docs/fase-4-cierre-orden.md), actualización
   de CLAUDE.md, docs/guia-estudio-src.md, docs/plan-de-trabajo.md y PDF.
FUERA DE ALCANCE (no hacer): pesaje final y conciliación (Fase 5); estado 4; reabrir una orden; campos
nuevos en el modelo (por ejemplo una fecha de cierre: la fecha y hora del cierre queda en
```

```
order_status_logs); frecuencia de almacenamiento y promedios; roles, alarmas por e-mail y frontend
(examen final); cambios en las Fases 1 a 3.

DECISIONES YA PROPUESTAS EN docs/plan-de-trabajo.md (29 a 32). Aplicalas tal cual; si Benjamín las
cambia, documentalo.
- D29: solo se cierra desde el estado 2. Los estados 1, 3 y 4 responden 409. Cerrar dos veces NO es
  idempotente: la segunda da 409.
- D30 (a confirmar por el equipo): el cierre no exige que la orden esté activada ni que tenga lecturas;
  alcanza con estar en estado 2. La consigna no dice otra cosa. Dejalo marcado en el PR y en el informe
  como decisión a confirmar.
- D31: la orden se busca con findByOrderNumberForUpdate para serializar el cierre con las lecturas.
- D32: una fila en order_status_logs (from 2, to 3, actor = usuario del token, observación fija "Orden
  cerrada para carga") y respuesta 200 con la orden (la misma representación que GET
  /api/v1/orders/{n}). No se modifican loadingStartDateTime, loadingEndDateTime ni los últimos valores.

=== ETAPA 0 — PREPARAR EL ENTORNO Y LA LÍNEA BASE ===
1. git status, git branch --show-current, git fetch. Confirmá que el árbol está limpio (salvo .postman/
   y postman/, que no se suben). Pasate a main, git pull --ff-only, y confirmá que incluye el PR #9 (git
   log --oneline).
2. Creá la rama: git checkout -b fase-4-cierre-orden. Si ya existe, no la pises: mostrale a Benjamín su
   estado y preguntale.
3. Verificá el entorno: Docker en marcha, contenedor iw3-gas-mysql arriba (docker compose up -d),
   usuario de prueba admin/admin123 sembrado (docs/sql/seed-dev-users.sql), puerto 8080 libre.
4. Medí la línea base ANTES de tocar código: ./mvnw -B test (esperado: 12 tests, 0 fallas) y la
   colección completa con newman 6 (esperado: 66 pedidos y 163 aserciones, 0 fallas). Si la línea base
   falla, frená y avisá.
   Recomendado para probar sin ensuciar iw3_db: crear una base descartable (docker exec iw3-gas-mysql
   mysql -uroot -proot -e "create database iw3_scratch"); arrancar la app en otro puerto con ./mvnw
   spring-boot:run y -Dspring-boot.run.arguments entre comillas con DOS argumentos: --server.port=8089 y
   --spring.datasource.url igual a la URL de src/main/resources/application.properties pero con la base
   iw3_scratch en lugar de iw3_db; sembrar el usuario de prueba en iw3_scratch una vez que la app
   arrancó (Hibernate crea las tablas); y correr newman con --env-var baseUrl=http://localhost:8089. Al
   terminar, bajá la app y borrá iw3_scratch.

=== ETAPA 1 — LECTURA Y PLAN (no modifiques código todavía) ===
Leé, en este orden: CLAUDE.md completo; docs/plan-de-trabajo.md (sección 6.1 y decisiones 29 a 32);
docs/pdf/consigna-trabajo-integrador-2026.pdf (página 3: Puntos 4 y 5; páginas 6 y 7: estados y notas
adicionales); docs/fase-3-carga-tiempo-real.md (informe y auditoría); docs/guia-estudio-src.md; y el
código: IOrderBusiness, OrderBusiness (registerTare, activate, logStatus), OrderDetailBusiness (add,
loadForUpdate), OrderRepository, OrderStatus, OrderStatusLog, OrderLoaderRestController,
OrderTmsRestController (molde de controller con getUserLogged), Constants, SecurityConfiguration, los
tests existentes y la colección Postman (carpeta 03 como molde). Si tenés a mano el material de la
cátedra (carpeta Ingenieria Web III ( Matio ), repositorio iw3_2026, clase 3 de REST y excepciones,
clase 7 de @Transactional, y IW3_Guia_Estudio_Clases.pdf), usalo solo como referencia de convenciones;
no lo modifiques. Si no lo tenés, seguí con el repo.
Después PRESENTALE A BENJAMÍN el plan de implementación: objetivo; archivos que vas a crear o modificar
(con la razón de cada uno); orden de las tareas T1 a T6; casos de prueba; riesgos (concurrencia cierre
vs lecturas, tests que dependan de la base); y las dudas que tengas. ESPERÁ su aprobación ("aprobado" o
ajustes) antes de modificar cualquier archivo.

=== ETAPA 2 — IMPLEMENTACIÓN (cada paso con commit atómico) ===
T1 — Negocio.
- IOrderBusiness: agregá close(Integer orderNumber, String actor) throws NotFoundException,
  ConflictException, BusinessException, con Javadoc en español (qué comprueba, qué cambia, qué escribe).
- OrderBusiness.close con @Override y @Transactional(rollbackFor = Exception.class). Orden de
  comprobaciones, TODAS FUERA del try para que el catch no las convierta en 500: (1) buscar la orden con
  orderDAO.findByOrderNumberForUpdate; si no existe, NotFoundException con el mismo texto que load ("No
  se encuentra la Orden numero=N"); el 404 va fuera del try; la consulta técnica sí va dentro de un try
  que la envuelve en BusinessException. (2) si el estado no es INITIAL_WEIGHING_REGISTERED,
  ConflictException con un mensaje que incluya el número de orden y el estado actual, por ejemplo "La
  orden N no se puede cerrar: debe estar en estado 2 (estado X)".
- Dentro del try: from = order.getStatus(); order.setStatus(CLOSED_FOR_LOADING); order =
  orderDAO.saveAndFlush(order) (saveAndFlush, no save: así un error de base sale dentro del try como
  BusinessException y no como 500 con formato de Spring; es la lección del hallazgo H-14);
  logStatus(order, from, order.getStatus(), actor, "Orden cerrada para carga"); devolver la orden. En el
  catch: log.error y BusinessException.builder().ex(e).build().
- Para el helper de búsqueda con bloqueo escribí uno privado equivalente al de OrderDetailBusiness
  dentro de OrderBusiness (no refactorices OrderDetailBusiness) y mencioná la duplicación en el informe.
- No toques loadingStartDateTime, loadingEndDateTime, los last* ni la contraseña.
```

```
T2 — Controller.
- OrderLoaderRestController: método close con @PostMapping(value = "/orders/{orderNumber}/close",
  produces = MediaType.APPLICATION_JSON_VALUE), SIN @RequestBody. Usá getUserLogged() como en
  OrderTmsRestController: actor = user == null ? null : user.getUsername().
- Respuestas: 200 con la orden (new ResponseEntity<>(order, HttpStatus.OK)); NotFoundException → 404;
  ConflictException → 409; BusinessException → 500, todas con response.build(status, e, e.getMessage()).
- Swagger: @Operation (summary y description en español: pasa la orden de 2 a 3, bloquea nuevas
  lecturas, registra el cambio en el historial) y @ApiResponse para 200 (con el esquema Order), 401,
  404, 409 y 500.
- No hace falta tocar Constants ni SecurityConfiguration (la ruta cuelga de URL_INTEGRATION_LOADER y ya
  exige JWT). Comprobá que sea así y no cambies nada que no haga falta.
T3 — Verificación del bloqueo de lecturas.
- Sin cambiar código: cerrá una orden y mandá una lectura. Esperado: 409 con el mensaje de
  OrderDetailBusiness ("no acepta lecturas ... estado 3"), y el histórico (GET
  /api/v1/orders/{n}/details) sin la lectura rechazada. Solo si esto falla, corregí OrderDetailBusiness
  con el cambio mínimo y explicalo.

=== ETAPA 3 — PRUEBAS (ejecutalas de verdad) ===
1. Tests unitarios (JUnit 5 + Mockito, que ya está disponible por las dependencias de test; sin cambiar
   el pom.xml): creá src/test/java/ar/edu/iua/iw3/business/OrderBusinessCloseTest.java con
   @ExtendWith(MockitoExtension.class), @InjectMocks OrderBusiness, @Mock OrderRepository y @Mock
   OrderStatusLogRepository. Casos mínimos: orden inexistente → NotFoundException; estado 1 →
   ConflictException; estado 3 → ConflictException; estado 4 → ConflictException; estado 2 → pasa a 3,
   se guarda y se registra UNA fila de log con from 2, to 3, el actor recibido y la observación; actor
   nulo o en blanco → el log queda con actor "sistema"; el cierre NO cambia loadingStartDateTime,
   loadingEndDateTime ni los last*. Corré ./mvnw -B test: tienen que pasar los 12 tests previos más los
   nuevos.
2. Postman: agregá a docs/postman/IW3-Gas-Terminal.postman_collection.json la carpeta "04 - Fase 4:
   cierre (sistema de carga)" SIN modificar las carpetas 00 a 03. Usá VARIABLES PROPIAS de colección
   (por ejemplo orderNumberClose, activationPasswordClose, orderNumberClosePending,
   orderNumberCloseBare) y copiá el patrón de la carpeta 03 para crear cada orden con un número propio.
   Respetá la sangría de 2 espacios y el fin de línea del archivo para que el diff sea limpio. Pedidos
   mínimos, con aserciones sobre fragmentos de mensaje (include), nunca sobre el texto completo:
   a. Crear orden (SAP), tara (TMS) y activación (sistema de carga) de la orden de cierre.
   b. Dos lecturas válidas (masas crecientes). Guardá en variables la cabecera (últimos valores,
      loadingStartDateTime, loadingEndDateTime) consultando GET /orders/{n}.
   c. Cerrar la orden: 200 y status 3.
   d. Consultar la orden: status 3; los last* y las fechas de carga IGUALES a las guardadas en (b);
      activationPassword ausente.
   e. Historial de estados (GET /orders/{n}/status-log): 3 registros; el último con from 2, to 3, actor
      "admin" y observación que incluya "cerrada".
   f. Lectura posterior al cierre: 409 y mensaje que incluya "no acepta lecturas".
   g. Histórico de lecturas (GET /orders/{n}/details): totalElements 2 (la rechazada no se guardó).
   h. Activar de nuevo la orden cerrada: 409.
   i. Cerrar otra vez la misma orden: 409 y mensaje que incluya "no se puede cerrar".
   j. Crear otra orden (queda en estado 1) y cerrarla: 409.
   k. Cerrar una orden inexistente: 404.
   l. Cerrar sin token (auth noauth): 401.
   m. D30: orden en estado 2 sin activar y sin lecturas (crear, tara, cerrar): 200 y status 3.
3. Corrida completa: newman 6 sobre TODA la colección. Esperado: las carpetas 00 a 03 siguen sin fallas
   y la 04 también. Informá cantidad de pedidos y aserciones (la línea base era 66 y 163; ahora serán
   más) y que no hubo fallas.
4. Concurrencia (verificá CA-6): con una orden activada y varias lecturas válidas, disparar en paralelo
   unas 50 lecturas de masa creciente y un cierre. Esperado: todas las respuestas son 200 o 409 (ningún
   500), el log tiene exactamente UNA fila 2→3, la orden queda en estado 3, ninguna lectura se guardó
   después del cierre (comparar reading_timestamp de order_details con la fecha del log 2→3), y
   last_accumulated_mass de la orden es igual al MAX(accumulated_mass) de sus filas en order_details.
   También: dos cierres simultáneos dan un 200 y un 409. Podés usar un script Node o PowerShell
   descartable fuera del repo, o curl en paralelo. Mostrale a Benjamín qué hiciste y qué obtuviste.
5. Casos límite a comprobar a mano y reportar: número de orden no numérico en la URL (el comportamiento
   por defecto de Spring); cierre con Content-Type JSON y cuerpo vacío; cierre de orden con y sin
   lecturas.
6. Limpieza: borrá los datos de prueba y la base descartable.

=== ETAPA 4 — DOCUMENTACIÓN Y EXPLICACIÓN DEL CÓDIGO ===
Benjamín tiene que poder defender esto oralmente, así que la documentación describe el CÓDIGO REAL, no
algo genérico.
1. Creá docs/fase-4-cierre-orden.md con las dos partes de siempre (mirá docs/fase-3-carga-tiempo-real.md
   como molde):
```

```
   - Informe técnico: inventario de archivos (archivo, acción creado/modificado/eliminado, motivo,
     responsabilidad); responsabilidad de cada clase afectada; función de cada método relevante
     (parámetros, retorno, validaciones, dependencias, errores); flujo de ejecución completo de POST
     .../close (controller → business → repositorio → base, con el orden de comprobaciones y el
     bloqueo); decisiones técnicas (D29 a D32, saveAndFlush, por qué el bloqueo pesimista, por qué no se
     agrega un campo de fecha de cierre); relación de cada cambio con la consigna (Punto 4 y notas de
     estados); pruebas ejecutadas con resultados reales; ejemplos concretos para probar (curl y
     Postman); limitaciones registradas.
   - Guía de estudio A–J: A objetivo, B conceptos teóricos (máquina de estados, transacciones, bloqueo
     pesimista, 409 vs 400; marcá qué es de la cátedra y qué es complementario), C arquitectura, D
     código paso a paso, E trazabilidad de datos, F ejemplo práctico, G al menos 10 preguntas del
     profesor, H respuestas modelo, I errores frecuentes, J lista de comprobación para rendir.
2. Actualizá docs/guia-estudio-src.md (el endpoint nuevo, OrderBusiness.close, el test nuevo).
3. Actualizá docs/plan-de-trabajo.md: Fase 4 como completada y verificada en la tabla de fases, y las
   decisiones 29 a 32 como "Aplicada en la Fase 4" (con la nota de que la 30 espera confirmación del
   equipo).
4. Actualizá CLAUDE.md: secciones 2 (estado), 5 (bloque "Fase 4"), 7 (fila del endpoint y cantidad de
   pruebas), 8.1 (Fase 4 completada), 9 (decisiones 29 a 32), 10 (hallazgos nuevos, si los hay) y 14
   (mapa de documentos). Corregí de paso las afirmaciones desactualizadas sobre la Fase 3 (dice que
   espera su PR, pero ya está en main con el PR #7 y las correcciones del PR #9). No toques la
   atribución de la decisión 11 de la Fase 3: es un pedido del equipo.
5. Swagger: confirmá en /swagger-ui.html (con el JWT) que el endpoint figura con sus respuestas.
6. PDF: generá docs/pdf/fase-4-cierre-orden.pdf y regenerá docs/pdf/guia-estudio-src.pdf con
   docs/pdf/md2pdf.js (instrucciones en docs/pdf/README.md; hace falta Node 18+ y Chrome, y CHROME_PATH
   si Chrome no está en la ruta por defecto). Agregá la fila del PDF nuevo en docs/pdf/README.md. Abrí o
   verificá que cada PDF se genera y tiene contenido.
7. Al final, dale a Benjamín en el chat un resumen en criollo de lo que hiciste, archivo por archivo, y
   cómo se prueba, para que lo estudie.

=== ETAPA 5 — REVISIÓN FINAL Y PUSH ===
Antes de publicar, hacé TODO esto y mostrale el resultado a Benjamín:
1. git status y git diff main...HEAD --stat: revisá el diff COMPLETO. Confirmá que no hay cambios ajenos
   a la Fase 4 (en particular, nada de OrderDetailBusiness, OrderDetailValidator ni las carpetas 00 a 03
   de Postman, salvo un arreglo mínimo y justificado), ni credenciales, ni secretos, ni archivos locales
   (.postman/, postman/, target/, node_modules, datos de prueba, archivos temporales).
2. git branch --show-current debe decir fase-4-cierre-orden. git fetch y confirmá que origin/main no
   trae cambios que choquen; si trae, integrá con git merge origin/main (sin reescribir historia) y
   volvé a probar lo afectado. No sobrescribas cambios de terceros.
3. Último chequeo de calidad: ./mvnw -B test en verde y newman completo en verde. NO publiques si hay
   errores bloqueantes en la compilación, las pruebas o la implementación: explicalos y frená.
4. Verificá que cada commit tenga ≤ 100 líneas, mensaje en español con el formato tipo(ámbito):
   descripción y SIN línea Co-Authored-By de Claude. Si algún commit quedó grande, partilo antes de
   publicar (sin reescribir lo que ya esté publicado).
5. Push de la rama de trabajo: git push -u origin fase-4-cierre-orden (nunca --force). Confirmá que
   terminó bien con git status y git log origin/fase-4-cierre-orden -1. Si el remoto, las credenciales o
   los permisos no están disponibles, informá el bloqueo con precisión y NO afirmes que publicaste.
6. No abras Pull Request ni mergees. Decile a Benjamín que, para integrar, el PR contra main lo abre él
   (o te lo pide), que por acuerdo del equipo (decisión 14) lleva la aprobación de 3 de los 4
   integrantes, y que la decisión 30 queda para que el equipo la confirme.

=== INFORME FINAL EN EL CHAT ===
Cerrá con un informe breve y verificable: (1) archivos creados, modificados y eliminados; (2) qué hace
cada pieza y cómo se prueba; (3) resultados reales: tests de Maven (cantidad y fallas), newman (pedidos,
aserciones, fallas), concurrencia (qué se disparó y qué respondió); (4) decisiones tomadas y pendientes
(D30); (5) lo que NO pudiste ejecutar, si hay algo; (6) el hash del último commit y la confirmación del
push, o el bloqueo que lo impidió. No declares la fase terminada si falta algún criterio de aceptación:
nombrá cuál falta.

=== CRITERIOS DE ACEPTACIÓN (la fase se da por terminada solo si se cumplen TODOS) ===
CA-1. Cerrar una orden en estado 2 responde 200 y la orden queda en estado 3 (también visto en GET
      /orders/{n}).
CA-2. order_status_logs suma una fila 2→3 con el usuario del token, fecha y hora y la observación.
CA-3. Una lectura enviada tras el cierre responde 409 y no se guarda (el histórico no cambia).
CA-4. Cerrar otra vez o cerrar una orden en estado 1 responde 409; orden inexistente 404; sin token 401.
CA-5. El cierre no cambia loadingStartDateTime, loadingEndDateTime ni los últimos valores.
CA-6. Con lecturas y cierre simultáneos no hay respuestas 500 y la cabecera coincide con el máximo
      guardado; dos cierres simultáneos dan un 200 y un 409.
CA-7. ./mvnw -B test y la colección Postman completa pasan sin fallas (las fases anteriores intactas).
CA-8. Swagger documenta el endpoint; existen docs/fase-4-cierre-orden.md con la guía (≥ 10 preguntas) y
```

```
      los PDF; CLAUDE.md, el plan y la guía de src están actualizados.
CA-9. Los cambios están publicados en la rama fase-4-cierre-orden (o el bloqueo está informado con
      precisión).

Empezá ahora por la ETAPA 0 y la ETAPA 1. No modifiques ningún archivo hasta que Benjamín apruebe el
plan.
```

8. Condiciones de entrega

Entrega	Condición
Código	Implementado según la sección 4; sin cambios ajenos a la Fase 4; compila y pasa `./mvnw -B test`
Pruebas	Tests unitarios nuevos, carpeta Postman 04, newman completo sin fallas y prueba de concurrencia, con resultados **reales** y registrados
Documentación	Swagger; `docs/fase-4-cierre-orden.md` (informe técnico y guía A–J con ≥ 10 preguntas); `docs/guia-estudio-src.md`, `CLAUDE.md` y `docs/plan-de-trabajo.md` actualizados; PDF en `docs/pdf/`
Explicación del código	Inventario de archivos, responsabilidad de cada clase y método, flujo de la petición, decisiones técnicas, relación con la consigna y ejemplos para probar; Benjamín debe poder defenderlo oralmente
Commits	Atómicos (≤ 100 líneas), mensaje en español `tipo(ámbito): descripción`, **sin** `Co-Authored-By` de Claude
Push	Rama `fase-4-cierre-orden`, sin `--force`, confirmado con `git log origin/fase-4-cierre-orden -1`. Si hay un bloqueo (remoto, credenciales, permisos), se informa con precisión y no se declara publicado
Integración	Pull Request contra `main` solo cuando Benjamín lo pida; requiere la aprobación de 3 de los 4 integrantes (decisión 14). La decisión 30 queda para confirmación del equipo
No subir	`.postman/`, `postman/`, `target/`, credenciales, datos de prueba y archivos temporales

Una fase está terminada cuando se cumplen todos los criterios de aceptación (CA-1 a CA-9), con pruebas **ejecutadas** y no solo escritas, las limitaciones registradas y la guía de estudio completa (regla 9 de `CLAUDE.md`).